

Resumen clase 18/08 - Introducción

¿Qué es un Sistema Operativo?

1. Proveedor de Abstracciones

2. Administrador de Recursos

3. Máquina Extendida

Historia de los sistemas operativos

Primera generación (1945 a 1955): tubos al vacío

 Características tecnológicas

 Capacidades y limitaciones

 Operación y programación

 Evolución hacia tarjetas perforadas

Segunda generación (1955 a 1965): transistores y sistemas de procesamiento por lotes

 Avances tecnológicos

 Infraestructura y operación

 Procesamiento por lotes

 Flujo típico de procesamiento por lotes

 Estructura de un trabajo típico

 Aplicaciones principales

Tercera generación (1965 a 1980): circuitos integrados y multiprogramación

Multiprogramación

Spooling

Timesharing

La cuarta generación (1980 a la fecha): las computadoras personales

 Avances tecnológicos

 Uso y accesibilidad

 Sistemas Operativos principales

 Características clave de esta generación

Interfaz con el usuario

 1. Interfaz de línea de comandos (CLI – Command Line Interface) - Shell

 2. Interfaz gráfica de usuario (GUI – Graphical User Interface)

Elementos básicos de un SO

Archivos

Directorio

Dispositivo virtual

Sistema de archivos

Directorio del sistema

¿Qué es un Sistema Operativo?

Un **sistema operativo** es un programa que administra el hardware de una computadora. También proporciona las bases para los programas de aplicación y actúa como un intermediario entre el usuario y el hardware de la computadora, proporcionándole a los programas de usuario un modelo de computadora mejor, más simple y pulcro.

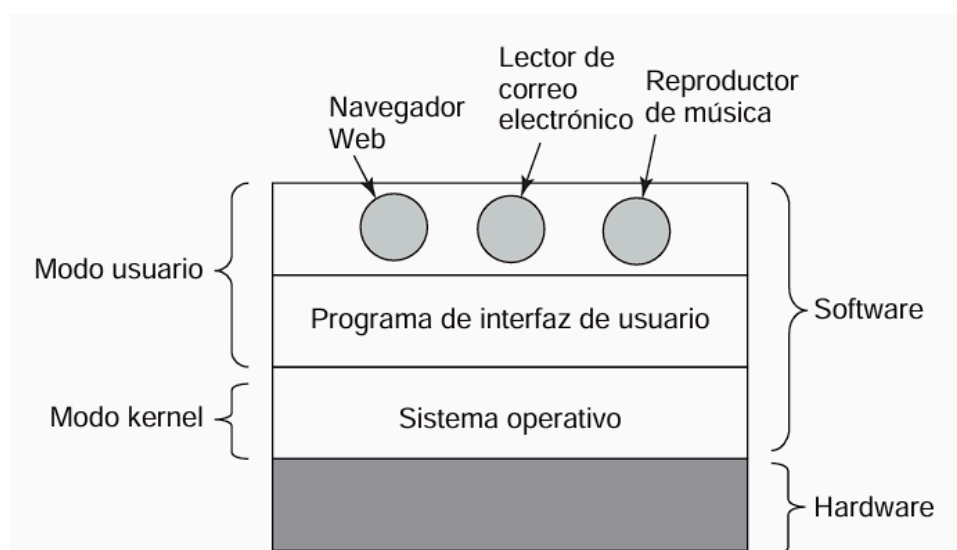


Figura 1-1. Ubicación del sistema operativo.

Un SO no es “solo un intermediario” — cumple **tres funciones esenciales** que explican por qué existe:

1. Proveedor de Abstracciones

El hardware en bruto es complejo:

- La CPU entiende únicamente **instrucciones de máquina**.
- La memoria es solo un conjunto de direcciones numéricas.
- Los discos son bloques de bits sin estructura.

- Los dispositivos de E/S tienen interfaces muy distintas entre sí.

👉 El SO oculta esa complejidad ofreciendo **abstracciones simples y uniformes**:

- **Proceso** → abstracción de la CPU.
 - Te hace creer que tenés "una CPU para vos solo".
- **Memoria virtual** → abstracción de la memoria física.
 - Cada proceso "cree" que tiene toda la RAM.
- **Archivos** → abstracción de los discos.
 - En lugar de leer bloques crudos, trabajás con `documento.txt`.
- **Dispositivo virtual** → abstracción de hardware de E/S.
 - Ejemplo: una impresora compartida aparece como si fuera exclusiva.

💡 Esta función es la que permite **portabilidad** y que los programas no tengan que saber los detalles de cada hardware.

2. Administrador de Recursos

El hardware es limitado (CPU, RAM, disco, red) y suele haber **muchos usuarios/procesos compitiendo**.

El SO actúa como **árbitro imparcial**, decidiendo:

- **CPU** → ¿qué proceso corre ahora? (Scheduling).
- **Memoria** → ¿qué proceso recibe cuánta RAM? (Asignación/paginación).
- **Disco** → ¿qué operación de lectura/escritura se atiende primero? (Políticas de E/S).
- **Red** → ¿qué proceso puede usar el socket, y con qué permisos?
- **Seguridad** → controlar accesos, permisos y privilegios.

Ejemplo: si dos procesos quieren imprimir al mismo tiempo, el SO **planifica y multiplexa** la impresora para evitar caos.

3. Máquina Extendida

El SO ofrece al programador un **modelo más sencillo y poderoso** que la máquina real:

- Una máquina real solo sabe ejecutar instrucciones binarias.

- Una máquina **extendida** (gracias al SO) ofrece **servicios de alto nivel**.

Ejemplo:

- Si querés escribir en pantalla, no tenés que manipular registros de video → usás la syscall `write()`.
- Si querés comunicarte en red, no manipulás bits eléctricos → usás sockets.

💡 Para el programador, la computadora **parece más rica y más fácil de usar** que el hardware desnudo.

En resumen:

- **Abstracciones** → hacen la vida más simple.
 - **Administración de recursos** → hacen que varios procesos puedan coexistir.
 - **Máquina extendida** → dan una computadora “más cómoda” que la real.
-

Historia de los sistemas operativos

Primera generación (1945 a 1955): tubos al vacío

Características tecnológicas

- **Componentes principales:** Tubos al vacío (bulbos), relevadores.
- **Ejemplos notables:**
 - *ABC* (Atanasoff-Berry Computer) – primera computadora digital funcional.
 - *Z3* – construida por Konrad Zuse en Berlín.
 - *Colossus* – usada para descifrar códigos en Bletchley Park, Inglaterra.
 - *Mark I* – desarrollada en Harvard por Howard Aiken.
 - *ENIAC* – creada por Mauchley y Eckert en la Universidad de Pennsylvania.

Capacidades y limitaciones

- Muy primitivas: tardaban varios segundos en realizar cálculos simples.

- Algunas eran binarias, otras programables, pero todas con capacidades limitadas.

Operación y programación

- Un solo grupo de personas se encargaba de todo: diseño, construcción, programación, operación y mantenimiento.
- Programación en lenguaje máquina o mediante *plugboards* (tableros de conexiones).
- No existían lenguajes de programación ni sistemas operativos.
- El proceso era manual y tedioso, con riesgo constante de fallos por bulbos quemados (¡hasta 20,000 por máquina!).

Evolución hacia tarjetas perforadas

- A principios de los años 50, se introdujeron las tarjetas perforadas.
- Permitían escribir programas en tarjetas en lugar de usar plugboards, aunque el resto del proceso seguía siendo igual de rudimentario.

Segunda generación (1955 a 1965): transistores y sistemas de procesamiento por lotes

Avances tecnológicos

- **Transistores reemplazan tubos al vacío**, mejorando la fiabilidad, tamaño y eficiencia energética.
- **Mainframes** como la IBM 7094 se vuelven accesibles para grandes instituciones.
- Separación clara de roles: diseñadores, operadores, programadores y técnicos de mantenimiento.

Infraestructura y operación

- Computadoras instaladas en **cuartos especiales con aire acondicionado**.
- Solo accesibles para **empresas grandes, universidades y gobiernos**.
- Programación en **FORTRAN o ensamblador**, usando **tarjetas perforadas**.

Procesamiento por lotes

- Se buscaba **reducir el tiempo muerto** de las máquinas.
- Se utilizaba una **computadora auxiliar (IBM 1401)** para:
 - Leer tarjetas perforadas.
 - Transferir trabajos a cinta magnética.
 - Imprimir resultados fuera de línea.
- La **computadora principal (IBM 7094)** ejecutaba los cálculos numéricos.

Flujo típico de procesamiento por lotes

1. Programador entrega tarjetas a la IBM 1401.
2. IBM 1401 transfiere trabajos a cinta magnética.
3. Cinta se lleva a la IBM 7094 para ejecución.
4. Resultados se guardan en otra cinta.
5. Cinta de salida se lleva de nuevo a la IBM 1401 para impresión.

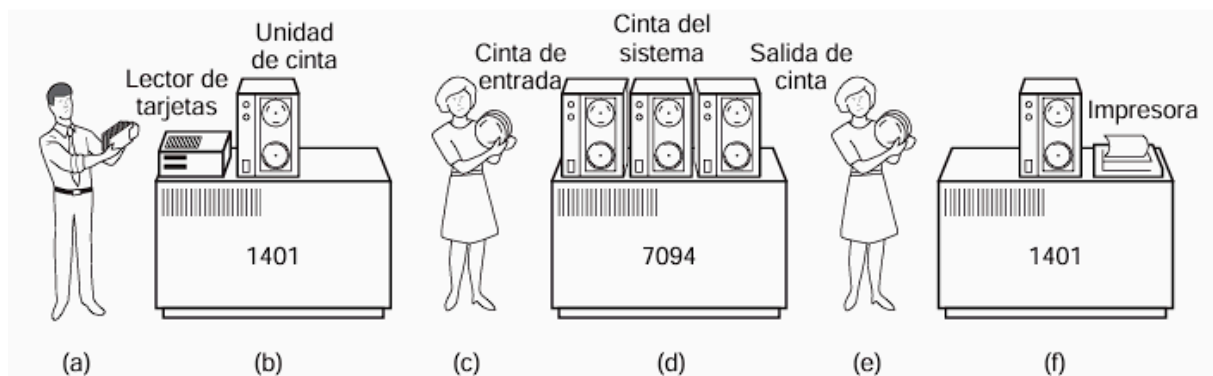


Figura 1-3. Uno de los primeros sistemas de procesamiento por lotes. a) Los programadores llevan las tarjetas a la 1401. b) La 1401 lee los lotes de trabajos y los coloca en cinta. c) El operador lleva la cinta de entrada a la 7094. d) La 7094 realiza los cálculos. e) El operador lleva la cinta de salida a la 1401. f) La 1401 imprime los resultados.

Estructura de un trabajo típico

- Tarjetas de control como:
 - **\$JOB** : inicio del trabajo.
 - **\$FORTRAN** : carga del compilador.
 - **\$LOAD** : carga del programa objeto.
 - **\$RUN** : ejecución del programa.

- **\$END** : fin del trabajo.

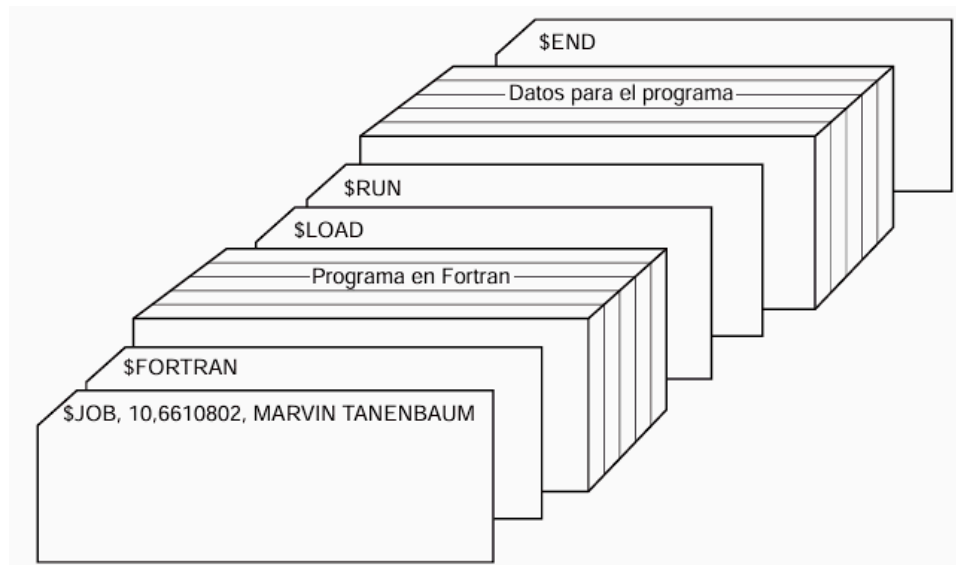


Figura 1-4. Estructura de un trabajo típico de FMS.

- Estas tarjetas fueron **precursoras de los shells modernos**.

Aplicaciones principales

- Cálculos científicos e ingeniería avanzada.
- Resolución de **ecuaciones diferenciales parciales**.
- Sistemas operativos destacados:
 - **FMS (Fortran Monitor System)**
 - **IBSYS** para IBM 7094.

Tercera generación (1965 a 1980): circuitos integrados y multiprogramación

Multiprogramación

Técnica clave que proporsionaba un uso del 100% de la maquina, ya que el problema era que cuando el trabajo actual se detenía para esperar a que se completara una operación con cinta u otro dispositivo de E/S, la CPU simplemente permanecía inactiva hasta terminar la operación de E/S, la solución fue particionar la memoria en varias piezas, con un trabajo distinto en cada partición. Mientras que un trabajo esperaba a que se completara una operación de E/S, otro podía estar usando la CPU.

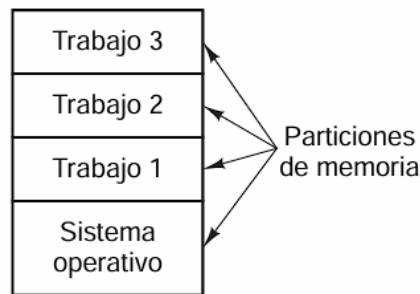


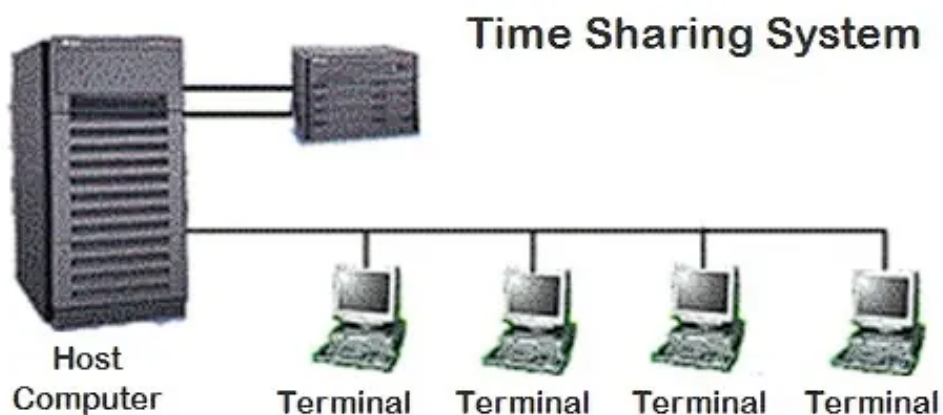
Figura 1-5. Un sistema de multiprogramación con tres trabajos en memoria.

Spooling

Spooling (de Simultaneous Peripheral Operation On Line, operación periférica simultánea en línea) fue la capacidad para leer trabajos en tarjetas y colocarlos en el disco tan pronto como se llevaban al cuarto de computadoras. Así, cada vez que terminaba un trabajo en ejecución, el sistema operativo podía cargar un nuevo trabajo del disco en la partición que entonces estaba vacía y lo ejecutaba también se utilizó para las operaciones de salida.

Timesharing

El tiempo compartido (timesharing), una variante de la multiprogramación donde cada usuario tenía una terminal en línea.



La cuarta generación (1980 a la fecha): las computadoras personales

Avances tecnológicos

- **Microprocesadores** → chip con CPU completa (Intel 8086, Motorola 68000).
- Discos duros más pequeños y económicos.
- Monitores, teclado y mouse como interfaz estándar.
- Redes locales (LAN) y luego Internet.

Uso y accesibilidad

- Las computadoras dejaron de ser exclusivas de gobiernos/empresas → llegaron a **usuarios individuales** y hogares.
- Auge del **software comercial** y los sistemas operativos de propósito general.

Sistemas Operativos principales

- **MS-DOS (1981)**: primera plataforma masiva de IBM PC.
- **MacOS (1984)**: pionero en **interfaces gráficas (GUI)**.
- **Windows (desde 1985, popularizado en 90s)**: GUI sobre DOS y luego SO independiente (Windows NT).
- **Linux (1991)**: kernel libre basado en UNIX, clave en servidores y luego en escritorios y móviles (Android).

Características clave de esta generación

- **Interfaz gráfica (GUI)** → ventanas, íconos, mouse → computación más accesible.
- **Multitarea real** en PCs → correr varios programas simultáneamente.
- **Redes e Internet** → los SO incorporan protocolos de red (TCP/IP).
- **Seguridad** comienza a ser un tema central (usuarios, contraseñas, antivirus).
- **Virtualización** y más tarde **contenedores (Docker)** como evolución de la compartición de recursos.
- **Cloud computing**: el SO ya no está solo en tu máquina, también “vive” en centros de datos.

Interfaz con el usuario

1. Interfaz de línea de comandos (CLI – Command Line Interface) - Shell

- **Descripción:** El usuario escribe comandos en texto para comunicarse con el sistema.
- **Ejemplos:** Bash en Linux, CMD o PowerShell en Windows.
- **Ventajas:**
 - Precisión y control total.
 - Ideal para tareas automatizadas y administración avanzada.
- **Desventajas:**
 - Requiere conocimiento técnico.
 - Poco amigable para usuarios novatos.

2. Interfaz gráfica de usuario (GUI – Graphical User Interface)

- **Descripción:** Usa ventanas, íconos, menús y punteros para facilitar la interacción.
 - **Ejemplos:** Windows, macOS, entornos gráficos como GNOME o KDE en Linux.
 - **Ventajas:**
 - Intuitiva y accesible.
 - Permite multitarea visual.
 - **Desventajas:**
 - Mayor consumo de recursos.
 - Menor precisión para tareas complejas.
-

Elementos basicos de un SO

Archivos

Un **archivo** es una colección de información relacionada que se almacena en disco. Puede contener datos de usuario (como documentos, imágenes, música) o información del sistema. Los sistemas operativos proporcionan abstracciones para que los archivos puedan ser manipulados sin preocuparse por los detalles físicos del disco.

- **Características clave:**

- Tienen nombre y atributos (como tamaño, fecha de creación, permisos).
- Se pueden abrir, leer, escribir y cerrar mediante llamadas al sistema.
- Son la unidad básica de almacenamiento persistente.

Directorio

Los **directorios** son estructuras que organizan y agrupan archivos. Funcionan como contenedores que permiten al usuario y al sistema operativo localizar y administrar archivos de manera jerárquica.

- **Tipos:**

- De un solo nivel: todos los archivos están en el mismo directorio.
- Jerárquicos: permiten subdirectorios, formando un árbol.

- **Operaciones comunes:**

- Crear, eliminar, renombrar, listar contenido.

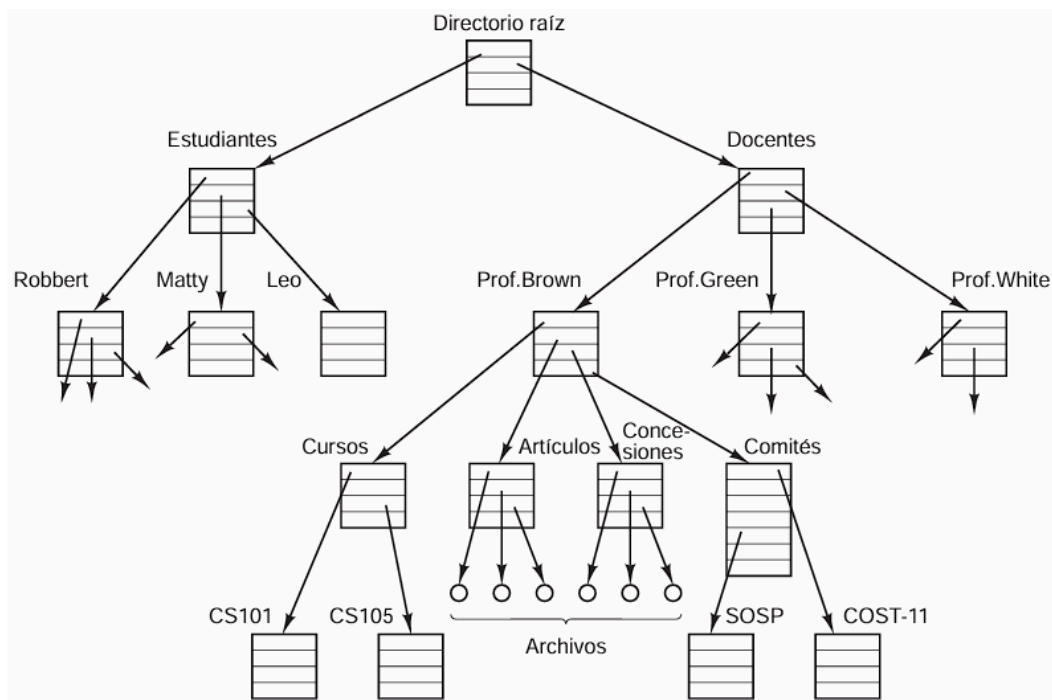


Figura 1-14. Un sistema de archivos para un departamento universitario.

Dispositivo virtual

Un **dispositivo virtual** es una abstracción que representa un dispositivo físico o lógico dentro del sistema operativo. Permite que múltiples procesos interactúen con recursos de hardware como si fueran independientes, aunque compartan el mismo dispositivo físico.

- **Ejemplos:**
 - Impresoras virtuales.
 - Interfaces de red virtuales.
 - Discos virtuales (como los usados por máquinas virtuales).
- **Función:** Facilitar la multiplexación y aislamiento de recursos.

Sistema de archivos

El **sistema de archivos** es el componente del sistema operativo encargado de administrar cómo se almacenan, organizan y recuperan los archivos en los dispositivos de almacenamiento.

- **Responsabilidades:**
 - Asignación de espacio en disco.
 - Mantenimiento de estructuras como tablas de archivos y directorios.

- Garantía de consistencia y seguridad.
- **Tipos:**
 - FAT, NTFS, ext4, entre otros.

Directorio del sistema

El **directorio del sistema** es una ubicación especial en el sistema de archivos que contiene archivos esenciales para el funcionamiento del sistema operativo.

- **Contenido típico:**
 - Ejecutables del sistema.
 - Bibliotecas compartidas.
 - Archivos de configuración.
- **Importancia:** Es crítico para el arranque y operación del sistema. Su corrupción puede impedir que el sistema funcione correctamente.

Binario del sistema

Un **binario del sistema** es un archivo ejecutable que forma parte del núcleo del sistema operativo o de sus utilidades esenciales. Está compilado en lenguaje máquina y puede ejecutarse directamente por el procesador.

- **Ejemplos:** `/bin/ls` , `/usr/bin/bash` en Linux; `explorer.exe` , `cmd.exe` en Windows.
- **Importancia:** Permiten la interacción con el sistema, desde comandos básicos hasta servicios críticos.
- **Ubicación típica:** Directorios como `/bin` , `/sbin` , `/usr/bin` .

Archivo de configuración

Un **archivo de configuración** contiene parámetros que definen el comportamiento de programas o del sistema operativo. Son esenciales para personalizar y controlar el entorno de ejecución.

- **Formato común:** Texto plano, con sintaxis específica (JSON, INI, XML, YAML).
- **Ejemplos:**
 - `/etc/passwd` (usuarios en Linux)
 - `httpd.conf` (configuración de Apache)

- `config.sys` (en sistemas DOS)
- **Función:** Permiten modificar sin recompilar el binario.

Usuarios

Los **usuarios** son entidades (personas o procesos) que interactúan con el sistema operativo. Cada usuario tiene un identificador único (UID) y puede tener permisos específicos.

- **Tipos:**
 - Usuario root o administrador.
 - Usuarios normales.
- **Gestión:** El sistema operativo controla acceso, permisos y recursos asignados.
- **Ejemplo en Linux:** listado en `/etc/passwd`.



No es necesario que un usuario corresponda directamente a una persona física

Grupos

Los **grupos** son colecciones de usuarios que comparten permisos o roles comunes. Facilitan la administración de acceso a archivos y recursos.

- **Ejemplo:** Grupo `sudo` permite ejecutar comandos como administrador.
- **Identificador:** GID (Group ID).
- **Ventaja:** Simplifica la gestión de permisos en entornos multiusuario.

Procesos

Un **proceso** es una instancia en ejecución de un programa. Es una de las abstracciones más fundamentales del sistema operativo.

- **Componentes:**
 - Código ejecutable.
 - Espacio de direcciones (memoria).
 - Estado (activo, suspendido, terminado).

- Process ID (PID)
 - **Gestión:** El sistema operativo asigna recursos, controla la planificación y sincronización.
 - **Ejemplo:** Cuando ejecutás `python script.py`, se crea un proceso con su propio espacio de memoria.
-

Resumen

- **SO = intermediario** entre usuario ↔ hardware.
- **Funciones:** proveer abstracciones, administrar recursos, extender la máquina.
- **Historia:**
 - 1ª gen (1945-55): tubos al vacío, sin SO, programación manual.
 - 2ª gen (55-65): transistores, procesamiento por lotes, tarjetas perforadas.
 - 3ª gen (65-80): circuitos integrados, multiprogramación, spooling, timesharing.
 - 4ª gen (80-): PCs, GUI, redes, Linux/Windows.
- **Interfaces:**
 - CLI (precisa, difícil).
 - GUI (amigable, más recursos).
- **Elementos básicos del SO:**
 - Archivos y directorios (almacenamiento jerárquico).
 - Sistema de archivos (FAT, NTFS, ext).
 - Dispositivos virtuales (impresoras, redes).
 - Binarios y config del sistema.
 - Usuarios (UID), grupos (GID).
 - Procesos (programa en ejecución, PID).
 - Kernel y syscalls = corazón del SO.